



CAP776

Class Assignment -4
PROGRAMMING IN PYTHON

Name: Jay Dev

Reg ID: 12322566

Roll No.:17

Section: DE328(G-1)

Daily Activity Data Collection and Analysis

Date (DD-MM-YYYY)

18-11-2024

Move Minutes(min)

Walking Duration(sec)

Calories (kcal)

Max Heart Rate(bpm)

Max Speed(m/s)

Distance (m)

Min Heart Rate(bpm)

Min Speed(m/s)

Step count

Avg Heart Rate(bpm)

Average Speed(m/s)

Add Data

Time Series Plot

Scatter Plot

Inactive Days

Heart Rate

Cumulative Analysis

Data loaded from CSV successfully!

Entries Loaded:71

Daily Activity Data Collection and Analysis

Date (DD-MM-YYYY)

18-11-2024

Move Minutes(min)

25

Walking Duration(sec)

20000

Calories (kcal)

2000

Max Heart Rate(bpm)

120

Max Speed(m/s)

2.3

Distance (m)

3000

Min Heart Rate(bpm)

60

Min Speed(m/s)

0.3

Step count

7000

Avg Heart Rate(bpm)

86

Average Speed(m/s)

0.7

Add Data

Time Series Plot

Scatter Plot

Inactive Days

Heart Rate

Cumulative Analysis

Data added successfully!

Total Entries:72

Daily Activity Data Collection and Analysis

Date (DD-MM-YYYY)

18-11-2024

Move Minutes(min)

25

Walking Duration(sec)

20000

Calories (kcal)

2000

Max Heart Rate(bpm)

120

Max Speed(m/s)

2.3

Distance (m)

3000

Min Heart Rate(bpm)

60

Min Speed(m/s)

0.3

Step count

7000

Avg Heart Rate(bpm)

86

Average Speed(m/s)

0.7

Add Data

Time Series Plot

Scatter Plot

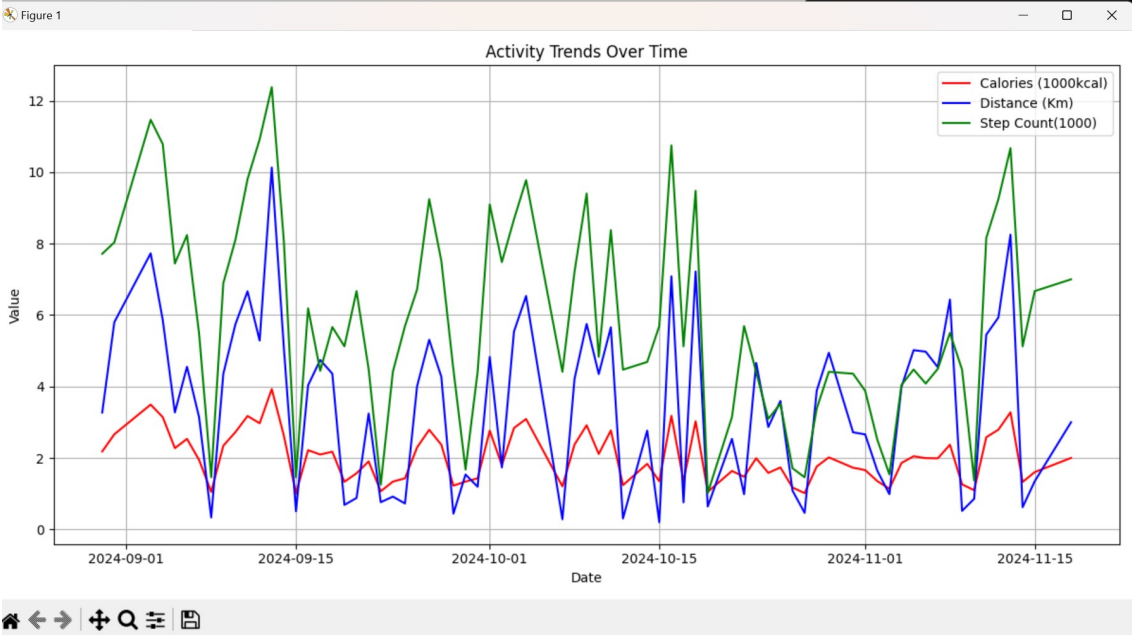
Inactive Days

Heart Rate

Cumulative Analysis

Data added successfully!

Total Entries:72



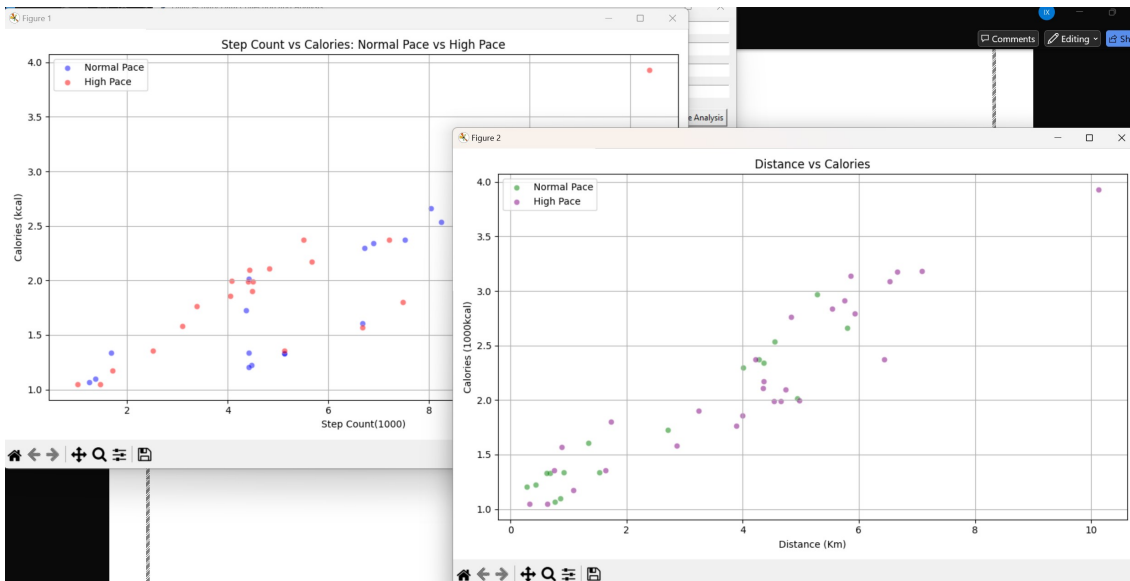
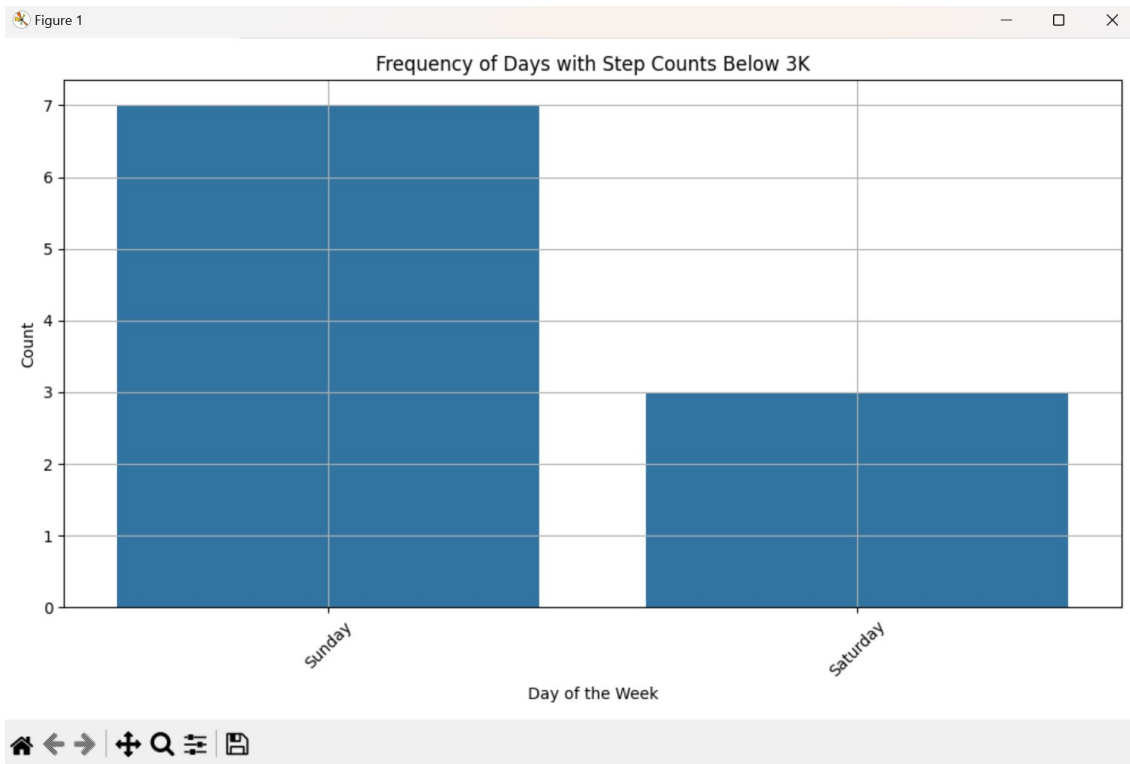


Figure 1

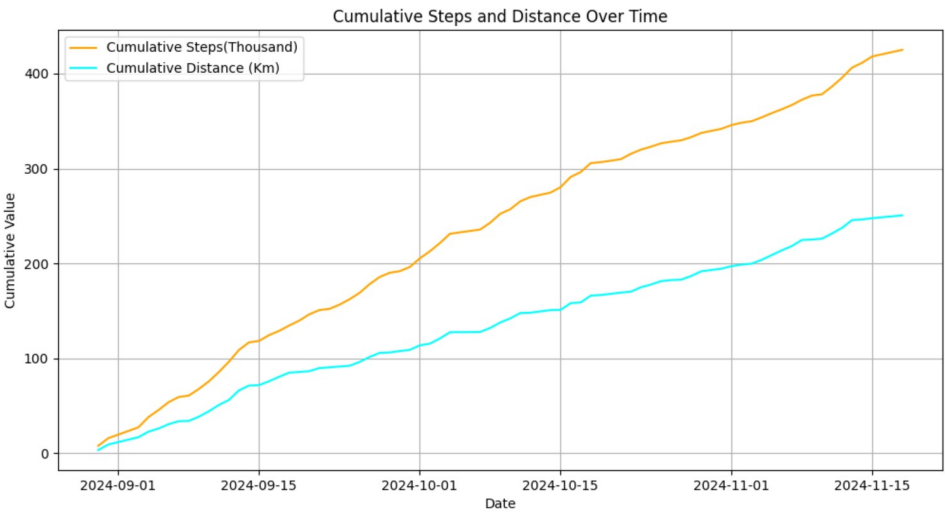
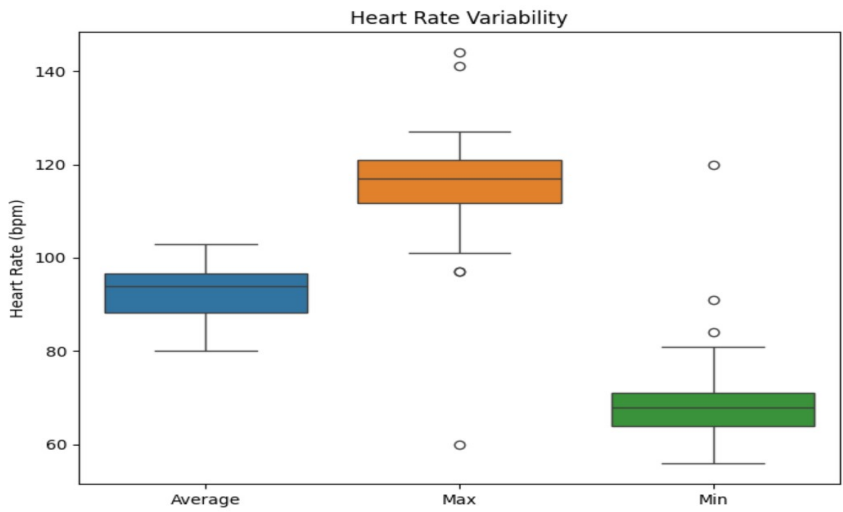


Figure 2



CODE

```
import tkinter as tk
from tkinter import messagebox
import pandas as pd
import seaborn as sns
from datetime import datetime
import matplotlib.pyplot as plt

data_file = 'Daily_Activity_metrics.csv'

# Initialize the main window
root = tk.Tk()

root.title("Daily Activity Data Collection and Analysis")
```

```
# Create a DataFrame to store the data
```

```
data = pd.DataFrame()
```

```
# Create a label to display messages
```

```
message_label = tk.Label(root, text="", fg="red")
```

```
message_label.grid(row=6, column=2, columnspan=2, padx=10, pady=5)
```

```
# Function to add data to the DataFrame
```

```
def add_data():
```

```
    global data
```

```
    date = pd.to_datetime(entry_date.get(), format='%d-%m-%Y')
```

```
    calories = entry_calories.get()
```

```
    distance = entry_distance.get()
```

```
    steps = entry_steps.get()
```

```
    move_minutes=entry_move_minutes.get()
```

```
    avg_heart_rate=entry_avg_heart_rate.get()
```

```
    max_heart_rate=entry_max_heart_rate.get()
```

```
    min_heart_rate=entry_min_heart_rate.get()
```

```
    max_speed=entry_max_speed.get()
```

```
    min_speed=entry_min_speed.get()
```

```
    average_speed=entry_average_speed.get()
```

```
    walking_duration=entry_walking_duration.get()
```

```
    if date and calories and distance and steps and move_minutes and avg_heart_rate and max_heart_rate and  
    min_heart_rate and max_speed and min_speed and average_speed and walking_duration:
```

```
        new_data = pd.DataFrame([[date, float(calories)/1000, float(distance)/1000, float(steps)/1000,  
float(move_minutes), float(avg_heart_rate), float(max_heart_rate), float(min_heart_rate), float(max_speed),  
float(min_speed), float(average_speed), float(walking_duration)]],
```

```
                                columns=['Date', 'Calories (1000kcal)', 'Distance (Km)', 'Step count(1000)', 'Move Minutes', 'Average  
heart rate (bpm)', 'Max heart rate (bpm)', 'Min heart rate (bpm)', 'Max Speed (m/s)', 'Min Speed (m/s)',  
'Average Speed (m/s)', 'Walking Duration (sec)'])
```

```
        data = pd.concat([data, new_data], ignore_index=True)
```

```
        message_label.config(text="Data added successfully!\nTotal Entries:"+str(data.shape[0]), fg="green")
```

```
    else:
```

```
        message_label.config(text="Please fill all fields", fg="red")
```

```
# Function to plot the data
```

```
def plot_data1():
```

```
    global data
```

```
    if not data.empty:
```

```
        plt.figure(figsize=(12, 6))
```

```
        sns.lineplot(x='Date', y='Calories (1000kcal)', data=data, label='Calories (1000kcal)', color='r')
```

```
        sns.lineplot(x='Date', y='Distance (Km)', data=data, label='Distance (Km)', color='b')
```

```
        sns.lineplot(x='Date', y='Step count(1000)', data=data, label='Step Count(1000)', color='g')
```

```
        plt.title('Activity Trends Over Time')
```

```
        plt.xlabel('Date')
```

```
        plt.ylabel('Value')
```

```
        plt.legend()
```

```
        plt.grid(True)
```

```
        plt.tight_layout()
```

```
        plt.show()
```

```
    else:
```

```
        message_label.config(text="No data to plot", fg="red")
```

```
def plot_data2():
```

```
    global data
```

```
    if not data.empty:
```

```
        # Define thresholds for normal and high pace
```

```
        normal_pace_threshold = 0.5 # m/s
```

```
        high_pace_threshold = 0.6 # m/s
```

```
        # Filter data for normal and high pace
```

```
        normal_pace_data = data[data['Average speed (m/s)'] < normal_pace_threshold]
```

```
        high_pace_data = data[data['Average speed (m/s)'] >= high_pace_threshold]
```

```
        # Plot normal pace data
```

```
        plt.figure(figsize=(10, 6))
```

```
        sns.scatterplot(x='Step count(1000)', y='Calories (1000kcal)', data=normal_pace_data, color='blue',  
        alpha=0.5, label='Normal Pace')
```

```
sns.scatterplot(x='Step count(1000)', y='Calories (1000kcal)', data=high_pace_data, color='red', alpha=0.5, label='High Pace')
```

```
plt.xlabel('Step Count(1000)')
```

```
plt.ylabel('Calories (kcal)')
```

```
plt.title('Step Count vs Calories: Normal Pace vs High Pace')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.figure(figsize=(10, 6))
```

```
sns.scatterplot(x='Distance (Km)', y='Calories (1000kcal)', data=normal_pace_data, color='green', alpha=0.5, label='Normal Pace')
```

```
sns.scatterplot(x='Distance (Km)', y='Calories (1000kcal)', data=high_pace_data, color='purple', alpha=0.5, label='High Pace')
```

```
plt.xlabel('Distance (Km)')
```

```
plt.ylabel('Calories (1000kcal)')
```

```
plt.title('Distance vs Calories')
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
else:
```

```
message_label.config(text="No data to plot", fg="red")
```

```
def plot_data3():
```

```
    global data
```

```
    if not data.empty:
```

```
        holiday_threshold=3 # 3K steps
```

```
        # Filter the dataframe for step counts below holiday threshold
```

```
        days_below_3000 = data[data['Step count(1000)'] < holiday_threshold]
```

```
        # Extract the day of the week for these days
```

```
        days_below_3000['Day of Week'] = days_below_3000['Date'].dt.day_name()
```

```
        # Get the frequency of each day of the week
```

```
        day_frequency = days_below_3000['Day of Week'].value_counts()
```

```
        # Convert the Series to a DataFrame and reset the index
```



```
day_frequency_df = day_frequency.reset_index()
day_frequency_df.columns = ['Day of the Week', 'Count']
```

```
plt.figure(figsize=(10, 6))
sns.barplot(x='Day of the Week', y='Count', data=day_frequency_df)
plt.xlabel('Day of the Week')
plt.ylabel('Count')
plt.title('Frequency of Days with Step Counts Below 3K')
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```

else:

```
message_label.config(text="No data to plot", fg="red")
```

```
def plot_data4():
```

```
    global data
```

```
    if not data.empty:
```

```
        plt.figure(figsize=(8, 6))
        sns.boxplot(data=data[['Average heart rate (bpm)', 'Max heart rate (bpm)', 'Min heart rate (bpm)']])
        plt.title('Heart Rate Variability')
        plt.ylabel('Heart Rate (bpm)')
        plt.xticks([0, 1, 2], ['Average', 'Max', 'Min'])
        plt.show()
```

else:

```
    message_label.config(text="No data to plot", fg="red")
```

```
def plot_data5():
```

```
    global data
```

```
    if not data.empty:
```

```
        data['Cumulative Steps(Thousand)'] = (data['Step count(1000)']).cumsum()
        data['Cumulative Distance (Km)'] = (data['Distance (Km)']).cumsum()
```

```
    plt.figure(figsize=(12, 6))
```

```
    plt.plot(data['Date'], data['Cumulative Steps(Thousand)'], label='Cumulative Steps(Thousand)',
             color='orange')
```

```

plt.plot(data['Date'], data['Cumulative Distance (Km)'], label='Cumulative Distance (Km)', color='cyan')

plt.title('Cumulative Steps and Distance Over Time')

plt.xlabel('Date')

plt.ylabel('Cumulative Value')

plt.legend()

plt.grid(True)

plt.show()

else:

    message_label.config(text="No data to plot", fg="red")

# Function to save data to CSV

def save_data():

    global data

    if not data.empty:

        data.to_csv(data_file, index=False)

        message_label.config(text="Data saved to CSV successfully!", fg="green")

    else:

        message_label.config(text="No data to save", fg="red")

def on_closing():

    if tk.messagebox.askyesno("Quit", "Do you want to save your data before quitting?"):

        save_data()

    root.destroy()

root.protocol("WM_DELETE_WINDOW", on_closing)

# Function to load data from CSV

def load_data():

    global data

    try:

        data = pd.read_csv(data_file)

        data['Date'] = pd.to_datetime(data['Date'])

        #comment if using new data

        data['Distance (Km)'] = data['Distance (m)'] / 1000

        data['Calories (1000kcal)'] = data['Calories (kcal)'] / 1000

        data['Step count(1000)'] = data['Step count'] / 1000

```

```
#comment if using new data
```

```
message_label.config(text="Data loaded from CSV successfully!\nEntries Loaded:"+str(data.shape[0]),  
fg="green")
```

```
except FileNotFoundError:
```

```
message_label.config(text="No CSV file found to load data", fg="red")
```

```
# Automatically load data when the application starts
```

```
load_data()
```

```
# Create and place the input fields and labels
```

```
tk.Label(root, text="Date (DD-MM-YYYY)").grid(row=0, column=0, padx=10, pady=5)
```

```
entry_date = tk.Entry(root)
```

```
entry_date.grid(row=0, column=1, padx=10, pady=5)
```

```
entry_date.insert(0, datetime.now().strftime('%d-%m-%Y'))
```

```
tk.Label(root, text="Calories (kcal)").grid(row=1, column=0, padx=10, pady=5)
```

```
entry_calories = tk.Entry(root)
```

```
entry_calories.grid(row=1, column=1, padx=10, pady=5)
```

```
tk.Label(root, text="Distance (m)").grid(row=2, column=0, padx=10, pady=5)
```

```
entry_distance = tk.Entry(root)
```

```
entry_distance.grid(row=2, column=1, padx=10, pady=5)
```

```
tk.Label(root, text="Step count").grid(row=3, column=0, padx=10, pady=5)
```

```
entry_steps = tk.Entry(root)
```

```
entry_steps.grid(row=3, column=1, padx=10, pady=5)
```

```
tk.Label(root, text="Move Minutes(min)").grid(row=0, column=2, padx=10, pady=5)
```

```
entry_move_minutes = tk.Entry(root)
```

```
entry_move_minutes.grid(row=0, column=3, padx=10, pady=5)
```

```
tk.Label(root, text="Max Heart Rate(bpm)").grid(row=1, column=2, padx=10, pady=5)
```

```
entry_max_heart_rate = tk.Entry(root)
```

```
entry_max_heart_rate.grid(row=2, column=3, padx=10, pady=5)
```

```
tk.Label(root, text="Min Heart Rate(bpm)").grid(row=2, column=2, padx=10, pady=5)
```

```
entry_min_heart_rate = tk.Entry(root)
```

```
entry_min_heart_rate.grid(row=1, column=3, padx=10, pady=5)
```

```
tk.Label(root, text="Avg Heart Rate(bpm)").grid(row=3, column=2, padx=10, pady=5)
```

```
entry_avg_heart_rate = tk.Entry(root)
```

```
entry_avg_heart_rate.grid(row=3, column=3, padx=10, pady=5)
```

```
tk.Label(root, text="Walking Duration(sec)").grid(row=0, column=4, padx=10, pady=5)
```

```
entry_walking_duration = tk.Entry(root)
```

```
entry_walking_duration.grid(row=3, column=5, padx=10, pady=5)
```

```
tk.Label(root, text="Max Speed(m/s)").grid(row=1, column=4, padx=10, pady=5)
```

```
entry_max_speed = tk.Entry(root)
```

```
entry_max_speed.grid(row=0, column=5, padx=10, pady=5)
```

```
tk.Label(root, text="Min Speed(m/s)").grid(row=2, column=4, padx=10, pady=5)
```

```
entry_min_speed = tk.Entry(root)
```

```
entry_min_speed.grid(row=1, column=5, padx=10, pady=5)
```

```
tk.Label(root, text="Average Speed(m/s)").grid(row=3, column=4, padx=10, pady=5)
```

```
entry_average_speed = tk.Entry(root)
```

```
entry_average_speed.grid(row=2, column=5, padx=10, pady=5)
```

```
# Create and place the buttons
```

```
tk.Button(root, text="Add Data", command=add_data).grid(row=4, column=0, padx=10, pady=10)
```

```
tk.Button(root, text="Time Series Plot", command=plot_data1).grid(row=4, column=1, padx=10, pady=10)
```

```
tk.Button(root, text="Scatter Plot", command=plot_data2).grid(row=4, column=2, padx=10, pady=10)
```

```
tk.Button(root, text="Inactive Days", command=plot_data3).grid(row=4, column=3, padx=10, pady=10)
```

```
tk.Button(root, text="Heart Rate", command=plot_data4).grid(row=4, column=4, padx=10, pady=10)
```

```
tk.Button(root, text="Cumulative Analysis", command=plot_data5).grid(row=4, column=5, padx=10, pady=10)
```

```
# Run the main loop
```

```
root.mainloop()
```

```
# save_data()
```